

The Alberta Buck - Rebalancing the BuckBasket (v1.0)

Waiting for the Turn: an Amortized On-Chain Rebalance Director

Perry Kundert

2026-07-08



The BuckBasket (see *The BuckBasket*) keeps itself roughly balanced for free: redemptions draw from overweight pools, deposits and treasury sweeps fill underweight ones, and the world's arbitrageurs do the rest. What that leaves on the table is *timing*. Basket constituents do not wander at random – they trail the M2 money supply with measurable, commodity-specific lags, so a constituent's share of basket value takes long, slow excursions away from its target weight and then comes home as the laggards catch up.

This article does four things. First, it measures those lags and turns them into a rebalancing policy: scale each constituent's rebalancing effort by how far it is from proportion, *times* how hard its delayed moving average is turning back toward proportion – quench while the excursion is still running, pour when it levels off. Second, it verifies the policy in simulation: the gated policies capture 75-90% of the continuous-rebalancing premium at a quarter to a half of the turnover, with by far the best per-trade timing.

Third, it changes coordinates. A single window on a single share-vs-target deviation harvests one timescale and carries the numeraire's noise; working instead in *differential mode* – pairwise log ratios, which cancel the common mode exactly, across a seven-scale filter bank – beats even continuous rebalancing's gross harvest, and is what deploys.

Fourth – the heart of the matter – it encodes all of this as an EVM state machine whose signal maintenance is *amortized across activations*: every deposit, redemption, or bare keeper trigger carries a small, bounded slice of the work, closed-form catch-up makes a poke after a quiet week cost the same as a poke after a busy hour, and the whole machine advises the basket through two $O(1)$ reads: where deposits should go, and which pool best satisfies a redemption.

1 The premise: money moves first, commodities follow

When the money supply expands, the new purchasing power does not reach all prices at once – the Cantillon insight. Monetary assets reprice in weeks; industrial commodities in months; wages and food over a year or more. For the BUCK basket this is measurable. Correlating each constituent's year-over-year growth against M2SL (post-1995 window, lags swept 0..36 months; see `alberta_buck/sim/quotes/fetch_feedstock.py` and `rebalance_policy.py --fit-lags`):

constituent	peak lag (months)	peak correlation	tier
cbBTC	0	0.08 (weak)	monetary (fast)
PAXG	censored @36	0.10 (weak)	monetary (prior)
CNST	9-10	0.48-0.52	industrial (mid)
NRGC	13	0.33-0.39	industrial (mid)
LABR	15-16	0.32-0.38	sticky (slow)
FOOD	23	0.21-0.26	sticky (slow)

The ordering is exactly the intuition: monetary, then energy and construction, then labour and food. (Gold's fit is weak – it trades on real rates and crisis as much as on M2 – so it takes a fast "monetary asset" prior rather than a fitted lag.)

The consequence for the basket: when M2 surges, the fast constituents overshoot their target share of basket value and stay overweight *until the laggards catch up* – an excursion with a timescale set by the lag spread, roughly a year. These excursions are the basket's harvest. A rebalancer that reacts to instantaneous deviation starts selling the riser immediately and keeps selling all the way up – paying fees and bleeding against the trend. One that waits for the excursion to *level off* makes fewer, larger trades at maximum mispricing.

2 The policy: quench while diverging, pour on the turn

Per constituent i , at a fixed sampling cadence (one epoch $\sim$ one day):

$\delta_i = w_i/w_i^* - 1$	– raw share deviation, right now;
$m_i = \text{EMA}_{X_i}(\delta_i)$	– the delayed view (X_i -day window);
$v_i = \Delta m_i$	– its velocity: receding / level / gaining;
$\dot{d}_i = \text{EMA}_{7d}(\Delta \delta_i)$	– the observed closure rate.

The rules, each of which was a *measured failure* in the Python model before it was a rule (`alberta_buck/sim/rebalance_policy.py`, make `nix-sim-policy`):

1. **Direction and magnitude come from the raw deviation; the MA contributes only timing.** A lagging average that still reads "overweight" after the raw share has reverted through target must not trade. Require `sign(m) == sign(delta)`.
2. **The velocity of the MA classifies the regime.** Still receding (deviation growing): do nothing – don't fight a move that is still developing. Level (turning): start, but not too fast. Gaining (closing): complete the remaining rebalancing.

3. **Size by rate-matching.** Trade at ρ times the closure rate the 1-week EMA actually observes – "the rate at which we'd lose half the distance to balance" – floored by a "half the gap in one window" prior (which alone applies while level):

$$\text{effort}_i = \min(\rho \hat{c}_i w_i^*, \text{cap}), \quad \hat{c}_i = \max\left(-\text{sign}(\delta_i) \dot{d}_i, \frac{\ln 2}{2X_i} |\delta_i|\right).$$

This is self-calibrating (no hand gain) and self-limiting: our own flow feeds the observed rate, so raising ρ saturates instead of overshooting. Effort peaks just after the turn – closure accelerating while deviation is still near maximum – and tapers as the gap closes.

4. **A leash bounds the quench.** A secular trend that never levels off (BTC, 2020-25) would otherwise run the deviation away unboundedly. Past

delta > 30% the policy trades at cap regardless of the gate, with

hysteresis release at 25%. The factor times trades *within* the leash; the leash enforces the mandate.

5. **Proceeds recycle, but never wash.** Sell-side flow re-enters the underweight pools (this is `sweepTreasury / investFromBucks`), but never a constituent the policy itself sold in the same round – a stale-signal sell paired with a raw-underweight rebuy just donates the AMM fee.

The mechanism, live on the 2020-25 history:

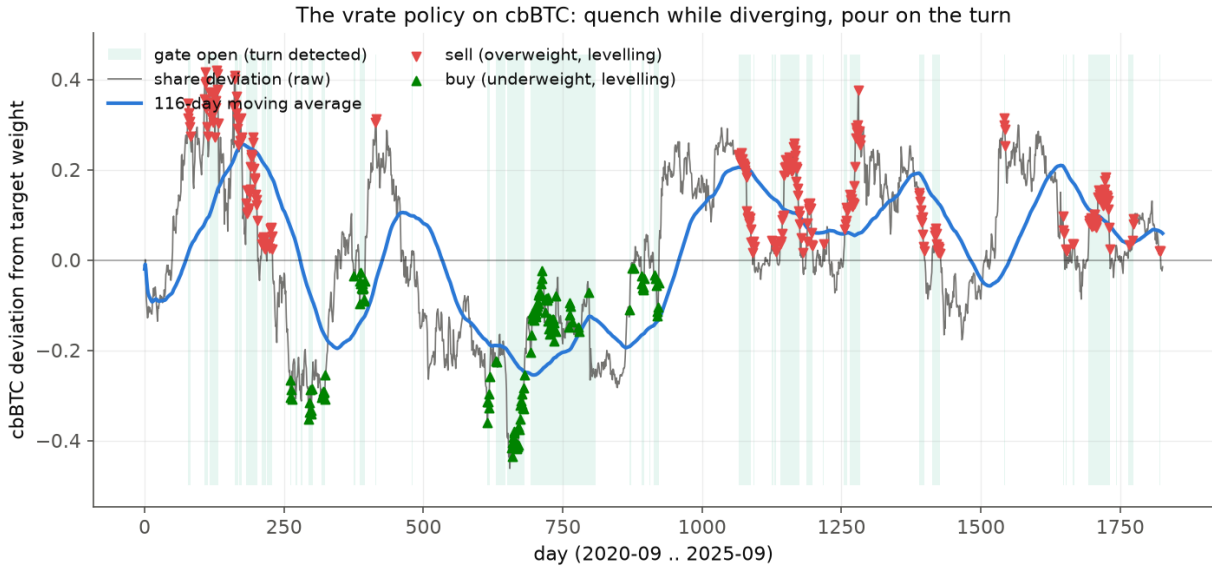


Figure 1: The gated policy on cbBTC's share of the basket, 2020-25. While the excursion is still building (the raw deviation pulling away from its moving average) nothing trades; sells (red) cluster where overweight excursions level off and turn, buys (green) where underweight ones do. Shaded bands mark the gate open.

Measured over 20-year synthetic paths with the fitted lags (cost 30bp/leg, cap 50bp NAV/day), against never rebalancing ("hold") and continuous proportional rebalancing ("prop", what spot-based routing approximates):

policy	premium vs hold	turnover	mean dev at trade	capture90
prop	+423 bp/yr	0.82 NAV/yr	5.3%	+219 bp
band (5%)	+411 bp/yr	1.42 NAV/yr	4.4%	+138 bp
factor (accel)	+381 bp/yr	0.44 NAV/yr	8.8%	+341 bp
vrate (rho=3)	+322 bp/yr	0.25 NAV/yr	12.6%	+514 bp

Each alternative loses to the gated policies on its own terms:

vs hold – doing nothing forfeits the entire rebalancing premium *and* abandons the mandate (weights drift wherever prices take them). Every active policy compounds steadily ahead of hold in the reverting regime:

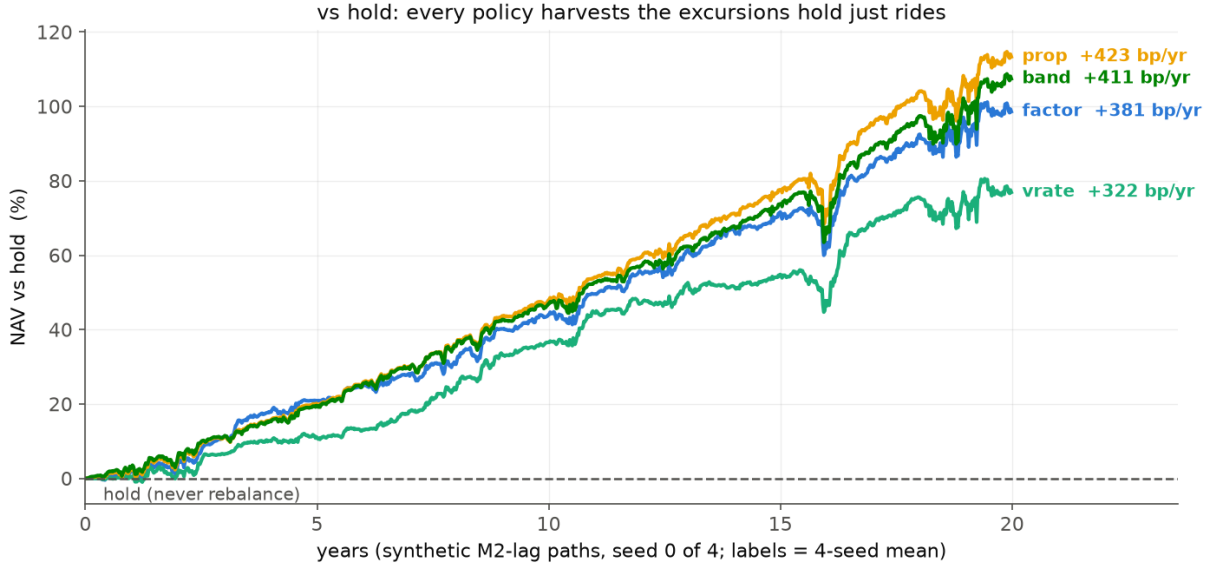


Figure 2: Twenty synthetic years, NAV relative to buy-and-hold (hold = the dashed zero line; line labels give the multi-seed mean premium). The premium is real and persistent – the only question is what it costs to collect.

vs prop (continuous proportional – what spot-based routing approximates) – prop collects the largest gross premium but pays for it everywhere: the most turnover (0.82x NAV/yr), the shallowest trades (5.3% mean deviation), and the worst behaviour in a secular trend, where it sells a riser continuously all the way up. The 2020-25 replay (BTC x9, reversion never arrives) is the stress case – the gated policies lose 90-130bp/yr *less* against the trend on a half to a third of prop’s turnover, while still holding the basket’s weights bounded:

vs band (the 5% threshold rebalancer) – band waits, but for the *wrong signal*: a fixed deviation level rather than the excursion’s turn. It re-engages the moment the band is crossed – early in a developing excursion – so it trades shallow (4.4% mean deviation), churns the most of any policy (1.42x NAV/yr, band-edge oscillation), and shows the worst per-trade timing. The gated policies wait for the turn itself and are strictly better on every per-trade measure:

In sum: the gated policies collect ~75-90% of prop’s premium at 30-55% of its turnover, trading at roughly twice the mispricing depth, with the least trend-bleed; full tables in `BASKET-REBALANCE.md`.

Every policy in that table reads one signal: a constituent’s share of basket value against its target, on a single window. The next section replaces that signal with a different one – and beats all four. It is the engine the deployed contract actually runs, so the table above is best read as the

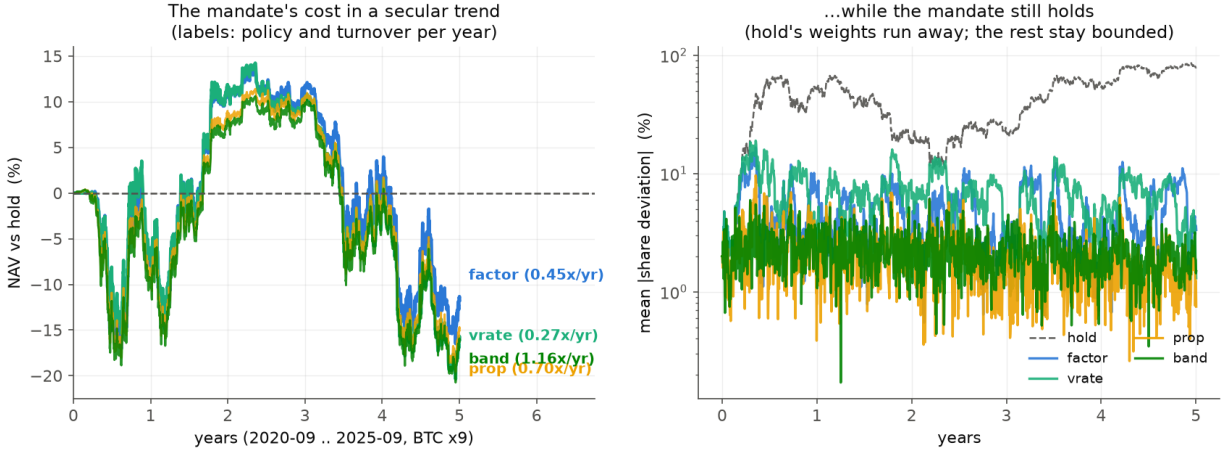


Figure 3: The mandate's cost under a secular trend (2020-25, BTC x9). Left: NAV against buy-and-hold – every constant-mix policy must sell the riser, but the gated policies (factor, vrata) bleed the least, on the least turnover. Right: buy-and-hold's mean absolute deviation runs off toward 80% – it has stopped being the declared basket – while every active policy keeps weights bounded.

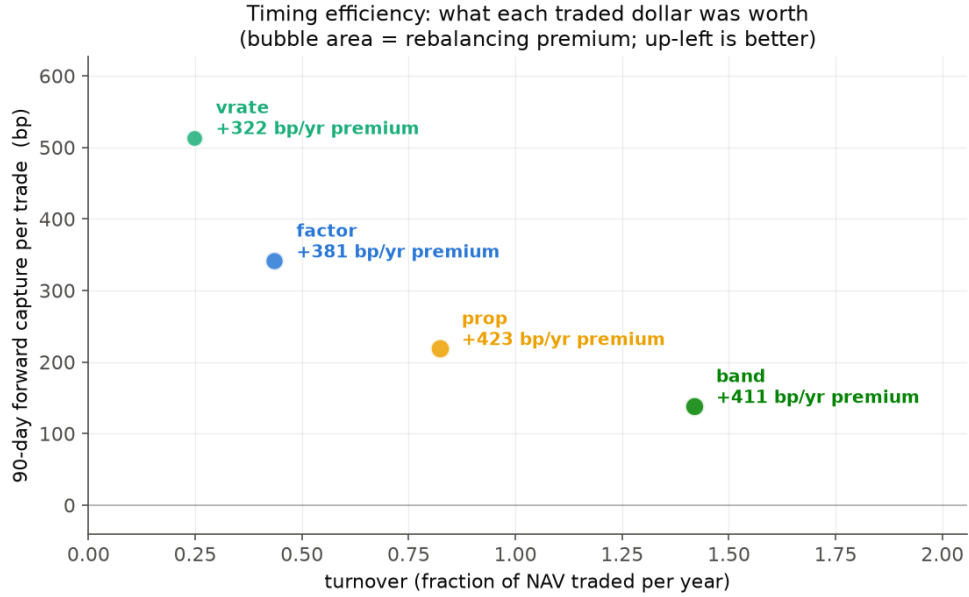


Figure 4: Timing efficiency on the synthetic paths: 90-day forward capture per traded dollar vs turnover (bubble area = premium vs hold). Band is dominated – more churn and worse timing than either gated policy; vrata captures +514bp per traded dollar at 0.25x NAV/yr of turnover.

argument for *gating*, not as the menu of what ships.

3 Differential mode: the basket as a polyphase machine

The policy above has two weaknesses, and both come from the *coordinate system* rather than from the gating.

First, **one window harvests one timescale**. Windows fitted to the M2 lag spread catch the year-long excursions and concede every mid-size swing to whoever is rebalancing continuously. Second, **share-of-basket-value is not a clean observable**. A constituent's share is measured from pool BUCK reserves, so it carries the numeraire's own noise: when BUCK moves against the dollar, every share moves, and the signal cannot tell that from a commodity actually getting rich.

The reconception (2026-07-11) is to change coordinates. In a three-phase machine one does not reason about the three line voltages; one splits them into a **common mode** (the zero sequence, which carries no torque) and the **differential modes** (which carry all of it). The basket has exactly this structure:

The profit lives in the differentials between commodity legs. The common mode – BUCK's own valuation against the numeraire – is the K-controller's job, and it **cancels exactly** in cross-commodity log price ratios.

So the **pairs** engine works pairwise and logarithmically. For legs i, j the imbalance is

$$d_{ij} = \ln \frac{1 + \delta_i}{1 + \delta_j},$$

which is numeraire-free by construction: any factor common to both legs subtracts out. A Uniswap *tick is already a log price*, so the engine reads ticks straight from `slot0` and never needs `ln()` on-chain.

3.1 A filter bank instead of a window

Each leg carries $K = 7$ EMAs of its normalized tick, on geometrically spaced windows (5, 10, 20, 40, 80, 160, 320 days). Because the EMA is linear, *every pair's moving average, velocity and curvature at every scale is the difference of two legs' ladders* – the full $N(N-1)/2$ differential graph falls out of $O(NK)$ state, with no pairwise storage at all.

Differences of adjacent EMAs are band-pass filters, so the ladder is an octave filter bank: the short windows see mid-size swings, the long ones see the M2 excursions, and the same gating logic applies independently at each scale.

3.2 The quorum turn detector

Per pair, each scale casts a vote, and it votes only when both of the single-scale rules hold at that scale: the MA gap agrees in sign with the raw pairwise imbalance, *and* its curvature points back toward equilibrium. The pair opens when votes reach a quorum (4 of 7 by default). No single scale can fire alone, which is what makes a fast window safe to include.

Effort is $\kappa |d_{ij}|$ (votes/ K), capped, under a pairwise leash with hysteresis (30% / 25%). Trades are **matched pairs** – sell the rich leg, buy the poor leg, equal value – so they are self-financing, leave no cash residue, and are structurally wash-proof: the engine cannot sell a constituent and rebuy it in the same round, which is the failure the single-scale policy needed an explicit no-wash rule to avoid. Progressive warmup lets the short windows vote from about day 20, so the engine is not silent for a 116-day window's warmup.

3.3 What it measures

Synthetic 20-year paths, 5 seeds, quorum 4, premium vs hold in bp/yr:

cost/leg	prop	band	factor	vrate	pairs
30bp	+430	+417	+393	+340	+496
100bp	+365	+308	+357	+313	+422
250bp	+228	+89	+283	+271	+262

pairs beats even *prop's gross harvest* – multi-scale coverage recovers the mid-size swings, and pair-matched execution harvests more per unit of imbalance – and dominates every policy through roughly 200bp/leg of cost. Only at extreme costs does **factor** retake the lead.

The quorum is the cost/premium knob: q3 maximizes premium (+508 at 1.39 turnover), q5 minimizes turnover (0.65 at +460), and **q4 is the knee**.

The vote criterion is the other knob, and it is the one that decides the engine's character (quorum 4):

vote	criterion	synth premium	turnover	capture90	hist CAGR
curv	divergence <i>decelerating</i> – early, at the top	+496	0.96	+201 bp	17.9%
vel	gap <i>already closing</i> – the confirmed turn	+421	0.60	+270 bp	18.4%

curv buys about 75bp/yr more gross harvest for 60% more turnover and worse trend bleed; the two cross at roughly 235bp/leg of cost. **vel** trades later and less, with better per-trade capture and prop-level trend robustness, and is **the deployment default** – it is the criterion `PairsRebalanceDirector.sol` implements.

3.4 Which engine, and when

The honest summary of the whole study is a regime split:

- **pairs is the reverting-regime pick** – the regime the *BuckBasket* is built for, and the one the physics argument in *The BuckBasket* says is the default behaviour of the physical economy.
- **factor remains the trend-regime pick** – best historical CAGR (19.3%) through the 2020-25 replay, where BTC ran x9 and reversion never arrived.

Between the two engines that *exist on-chain*, though, the choice is not close: **pairs** dominates **vrate** on both the premium and the risk measure. **vrate** concentrates its trades where observed closure is fastest, which in a trend means selling into sharp V-shaped pullbacks that then resume – its historical capture90 is the worst of any policy studied at -676bp, against **pairs** / **vel** at +270bp. In the reverting regime that same concentration is maximally right (+514bp, the best of any policy), which is precisely what makes it the wrong default: it is the policy whose performance depends most on the regime being the one you hoped for.

4 Encoding it on-chain: work that amortizes across whoever shows up

The policy needs a clock – moving averages advance with time – but the EVM has no scheduler and nobody pays to run one. The established answer is the pattern behind Compound's `accrueInterest`, MakerDAO's per-collateral `drip`, and Uniswap's oracle observations:

Lazy time-indexed accrual. State is advanced on first touch, as a closed-form function of elapsed time – never per block.

Three refinements make it fit a *multi-pool signal aggregator*:

4.1 Closed-form EMA catch-up

Under sample-and-hold (the deviation observed now stands in for the quiet gap), n missed epochs of EMA recursion collapse exactly, with $q = 1 - \beta$ (and q_7 the one-week EMA's decay):

$$\begin{aligned} m' &= \delta + (m - \delta) q^n && \text{– the average;} \\ v' &= q^n v - (1 - q)^2 n q^{n-1} (m - \delta) && \text{– its velocity;} \\ \dot{d}' &= q_7^{n-1} ((\delta - \delta_{\text{prev}})(1 - q_7) + q_7 \dot{d}) && \text{– the closure rate;} \end{aligned}$$

each computable in $O(\log n)$ by binary exponentiation. One poke after a quiet month costs the same as one poke after a busy hour, *and lands the signals in the same place* n diligent pokes would. This is not approximately true; it is an invariant the test suite fuzzes (256 random gap/imbalance combinations) and enforces to fixed-point rounding. (The velocity closed form matters: a naive "average dm over the gap" EMA update classifies the regime differently than the diligent recursion – a real bug the regime test matrix caught.)

4.2 The round-robin work wheel

N constituents, a persistent cursor, and `poke(maxWork)`: each call advances at most `maxWork` stale constituents from the cursor on. Every basket activation – deposit, redeem, treasury sweep, or a bare keeper trigger – carries a small budget (1-2). The gas story, measured (6-token mocks, via-IR, solc 0.8.28):

operation	gas
poke one constituent, 1-epoch gap	~36,500
poke one constituent, 50-epoch gap	~19,900
poke when everything is fresh	~3,100

Catch-up is *flat in gap size* (the 50-epoch poke is cheaper than the 1-epoch one purely from storage warmth), and the fresh-epoch guard is near-free. So: **the more granular the triggering, the less work each agent performs** – and if nobody shows up for a week, whoever arrives next pays one closed-form update, not a week of them. Work is conserved; granularity trades per-call gas against signal staleness, never total cost.

4.3 Cached observations with running sums

A constituent's deviation needs *everyone's* pool values (its share of the total). Reading N pools per poke would defeat the sharding, so each poke reads only *its own* pool – one `balanceOf` (the BUCK reserve bv_i ; for full-range positions pool value = $2bv_i$) and one `slot0` (spot, for the price-scaled target share $s_i = a_i P_{0,i}^2 / \text{spot}_i$, the same fixed-quantity-index target the pro-rata redemption uses) – and folds the result into cached running sums by difference:

$$B += bv_i^{\text{new}} - bv_i^{\text{cached}}, \quad S += s_i^{\text{new}} - s_i^{\text{cached}}, \quad \delta_i = \frac{bv_i/B}{s_i/S} - 1.$$

Deviations are computed against slightly-stale sums – which a moving-average policy is insensitive to by construction. Decision data aggregates the same way: each poke stores its constituent’s signed advisory effort, and the two decision reads – `depositHint()` (strongest buy-side effort) and `redeemHint()` (strongest sell-side) – are trivial scans over N cached `int32`s.

4.4 The trust model: hints, not authority

The director holds no funds and executes nothing. Its outputs are *advisory*: the basket’s venue re-verifies everything material (the spot-vs-TWAP manipulation guard, liquidity bounds, the conversion-loss budget) on every trade it actually performs. A stale, wrong, or even adversarial hint can re-order flows – route a deposit to the second-best pool – but cannot break solvency or extract value beyond what the venue’s own guards already bound. That is what makes it safe for the signal state to be lazily maintained by unauthenticated public pokes.

5 The contracts: two engines on a shared chassis

`RebalanceDirectorBase.sol` is the policy-agnostic chassis – epoch clock, `poke(maxWork)` work wheel, metadata sync, cached observations with running sums, binary exponentiation, and the `depositHint` / `redeemHint` / `effortOf` surface – and two signal engines plug into it behind the common `IRedirector` advisory interface:

- `BasketRebalanceDirector.sol` (**vrate**) – the single-window share-vs-target policy derived above.
- `PairsRebalanceDirector.sol` (**pairs**) – differential mode: per-leg 7-window tick-EMA ladders, pair quorum votes on the confirmed turn (**vel**), arithmetic weight-ratio differential, pairwise leash bitmask, and `bestPair()` naming the top matched trade for keepers. Pair efforts are folded into per-leg *net* efforts at poke time, so the chassis’s advisory reads stay $O(N)$ cached – fully API-compatible with **vrate**.

pairs at quorum 4 is what deploys, and what the rebalancing simulation runs. The rest of this section describes **vrate**, whose state layout is the simpler illustration of the shared machinery. `src/basket/BasketRebalanceDirector.sol` (~2.5 slots of hot state per constituent, packed):

```
struct Signal {
    int128 m;           // EMA_X of deviation (1e18)
    int128 vEma;        // EMA_X of per-epoch d(m)
    int128 ddotEma;     // 1-week EMA of per-epoch d(delta)
    int128 prevDelta;
    uint128 bv;         // cached pool BUCK reserve
    uint128 s;          // cached price-scaled target share
    int32 effortBp;     // signed advisory effort (bp of NAV per epoch)
    uint32 lastEpoch;
    uint32 firstEpoch;
    bool   leashed;
}
```

The API surface:

- `syncConstituents()` – permissionless; pulls constituent metadata from the basket’s public getters (re-callable: `addBasketToken` renormalizes).
- `poke(maxWork)` / `pokeAll()` – the work wheel. Skips fresh constituents for ~ 2 warm SLOADs; advances stale ones with the closed forms above and the vrate effort rule (regime gate, rate-matched size, leash, warmup).
- `depositHint()` / `redeemHint()` / `effortOf(i)` / `effortsAll()` – the $O(1)/O(N)$ -cached advisory reads. A `type(uint256).max` hint means "no signal; use the default most-underweight routing".
- `setParams(...)` – governance: epoch length, window, ρ , deadband, regime band, leash bounds, per-epoch cap.

How `BuckBasketProRata` consumes it (the integration increments, in order of invasiveness):

1. **Increment 0 – flow routing, zero new trades.** `_depositBuck` and `sweepTreasury` pass `director.depositHint()` as the `poolHint` to `investFromBucks` (falling back to most-underweight on the sentinel), and the single-token redeem path consults `redeemHint()`. Every such activation also calls `director.poke(1)` – the bounded slice that keeps the signals alive. The basket’s ordinary deposit/redeem/sweep traffic then performs well-timed rebalancing *for free*.
2. **Increment 1 – keeper rebalance steps.** A permissionless pair, `rebalanceSource(i)` (withdraw + convert from the strongest sell-side pool, bounded by `effortBp`) and `rebalanceSink()` (invest the pending BUCK into the deposit hint), each one small transaction – the `sweepTreasury` pattern, directed by the signals.
3. **Weighted draws.** `_allocateSellHigh` can tilt its per-pool draw weights by `effortsAll()` rather than spot overweight alone.

6 Verification

Three layers, all runnable from the Makefile:

(The model figures above regenerate via `make nix-sim-policy` then `make nix-sim-plot-article`.)

- **Unit + fuzz (Forge)** – `make nix-test-director`. Ten tests: the amortization invariant (diligent-vs-lazy twins, plus the 256-run fuzz over random gaps and imbalances), regime transitions on a scripted excursion (receding quenched, level starts, gaining saturates, sign-agreement blocks the stale-MA wrong-way trade), leash engagement and hysteresis, both hints, wheel budgets and cursor rotation, and the gas measurements quoted above.
- **Parallel regime matrix** – `make nix-test-director-regimes`. The same suite re-run across a window x rho grid (defaults: $\{6, 8, 16\}$ epochs x $\{1.5, 3, 6\}$), one forge process per regime, in parallel. This matrix is what exposed the velocity catch-up approximation: windows of 6 classified a post-gap regime differently between the lazy and diligent paths until the closed form replaced the average-velocity shortcut.
- **Anvil end-to-end** – `make nix-sim-director`. The rebalancing scenario (~ 400 agents: arbs, LPs, direct-mint depositors, a whale) plus a `DirectorKeeperAgent` that pokes the deployed

director with a 2-constituent budget each tick and executes its advice as 2-hop Universal Router swaps (sell-hint -> BUCK -> buy-hint), sized by the advised bp of basket NAV. The output vector records **directorPokes** / **directorTrades** per day alongside the existing basket telemetry.

7 Variants worth investigating

Nothing in this section is measured. It is the set of directions the machine's own structure suggests, each with the literature that would supply the theory, ordered by what looks like the best return on effort. Two analogies organize them, and both are load-bearing rather than decorative.

7.1 Taking the machine analogy literally

The three-phase intuition that produced differential mode has more in it. A polyphase drive does not stop at "separate the common mode"; it applies two specific transforms, and both have an exact basket counterpart:

- The **Clarke transform** (abc to alpha-beta-zero) splits a three-phase vector into a zero sequence – which carries no torque – and two *orthogonal* differential components. The basket's log-weight vector splits the same way: one common mode (BUCK against the numeraire, the K-controller's business) and $N - 1$ differential modes.
- The **Park transform** (alpha-beta to d-q) then rotates into a frame turning with the field, so that quantities which were sinusoidal become constant and a plain PI controller suffices. Field-oriented control splits current into a torque-producing component and a flux-producing one, and regulates them separately.

The basket's d-q split is the useful part: the component of an imbalance that is *harvestable* (mean-reverting – torque) versus the component that is *secular* (a trend that must be tolerated and bounded – flux). The leash is today a crude version of exactly this distinction, applied after the fact and with a hand-set bound.

And **slip** is the concept the engine already rediscovered. Induction torque is proportional to slip – the *relative* rate between field and rotor – and vanishes at zero slip, however fast both are turning. That is precisely the rate-matching rule: harvest scales with the rate of relative movement, not with absolute price motion. The machine analogy retroactively validates the policy's central choice.

7.2 Taking the frequency analogy literally

The EMA ladder is already a temporal decomposition: geometrically spaced windows, differences of adjacent EMAs acting as band-pass filters – an octave filter bank, which is a wavelet transform under another name. What is *not* decomposed is the cross-sectional axis. The full object is two-dimensional – **mode x timescale** – and the engine currently does one axis properly and the other by brute force.

7.2.1 1. Orthogonal modes instead of pairs

The sharpest available critique of the **pairs** engine is that its basis is redundant. Six legs have **five** independent differential modes, but the engine tracks **fifteen** pairs. The quorum therefore counts

heavily overlapping evidence as if it were independent, and effort sizing double- and triple-counts the same underlying excursion.

The fix is the Clarke transform generalized: diagonalize the covariance of log log-returns and work in the eigenbasis. PC1 is the common mode – which should be *discarded* here, because it is the controller’s job – and PC2..PCN are orthogonal, so each can be sized independently without double-counting, and a quorum over modes is a quorum over genuinely independent evidence. This is the standard construction in statistical arbitrage (Avellaneda and Lee, *Statistical Arbitrage in the US Equities Market*, 2010: PCA eigenportfolios with mean-reverting residuals).

Cost on-chain is an $N \times N$ fixed-point rotation applied to the cached ladders. At $N = 6$ that is small, and the eigenvectors are governance parameters refitted rarely – not per poke. The cointegration literature (Johansen) gives the more principled version still: for constituents bound to each other by physics, the object to trade is the *stationary* linear combination, not the pairwise raw imbalance.

7.2.2 2. Trade to the boundary, not to the target

Under proportional transaction costs the optimal policy is provably **not** to trade toward the target: it is a **no-trade region**, and one trades only enough to return to its boundary (Davis and Norman 1990; Shreve and Soner 1994). The current engine sizes effort toward proportion and relies on the cap and deadband to stop it overshooting. Reformulating the deadband as a genuine no-trade wedge, with effort sized to reach the near edge rather than the centre, should cut turnover with no loss of premium – the cheapest change on this list, and the one with the firmest theory behind it.

7.2.3 3. Bands fitted to the reversion, not set by hand

The deadband is a flat 150bp and the leash a flat 30%. For a mean-reverting spread the optimal entry and exit thresholds are known in closed form as a function of the reversion rate, the volatility and the cost (Bertram 2010; Leung and Li on optimal Ornstein-Uhlenbeck trading), and they scale like $\sigma/\sqrt{2\theta}$. Since the M2-lag study already fits a timescale per constituent, fitting (θ, σ) per *pair* or per *mode* is the same kind of exercise, and it replaces two hand-set constants with measured ones.

7.2.4 4. Weights are bounded, so bound them properly

"Strict upper and lower bounds" has an exact stochastic counterpart: weights live on a simplex, and the natural diffusion there is the **Jacobi (Wright-Fisher)** process, whose stationary distribution is Beta and whose first-passage behaviour is known. Fitting it gives the leash a *quantile* instead of a guess – "this excursion is at the 99th percentile of what this constituent’s share does" is a defensible trigger in a way that "30%" is not.

7.2.5 5. A target-share family with a theorem attached

Constant-mix is one point in a family. Fernholz’s stochastic portfolio theory shows that a portfolio generated by a concave function of market weights decomposes into a generating term plus a drift term that is *non-negative*, and that **diversity-weighted** portfolios ($w_i \propto \mu_i^p$, $0 < p < 1$) outperform the market portfolio under a diversity condition – which is exactly the bounded-weights premise the BuckBasket rests on. The rebalancing premium this article measures *is* Fernholz’s excess growth rate. Exposing p as a governance parameter gives a continuous knob from equal-weight (maximum premium, maximum turnover) to cap-weight (zero of both), with a proof rather than a simulation behind the interpolation.

7.2.6 6. Phase, not just velocity

"Wait for the turn" is a statement about *phase*: trade when an excursion is near the end of its cycle. The engine currently infers this from the sign of a velocity or curvature, which is a one-bit reading of a continuous quantity. The analytic-signal construction (a Hilbert transform, cheaply approximated by an EMA quadrature pair) yields instantaneous amplitude and phase directly, so effort could be a smooth function of how far through the excursion a pair actually is.

For genuine periodicity – and energy and food have real seasonality – the **Goertzel algorithm** computes a single DFT bin with two state variables and $O(1)$ work per sample, which is the one FFT-family primitive cheap enough to run on-chain unmodified. Worth it only if a fitted seasonal component turns out to be material; worth *testing* for that reason.

7.3 What not to do

Two attractive-sounding directions are traps, and the article's own data says so. **Do not chase gross premium**: `curv` and `quorum 3` both buy more harvest with more turnover, and the whole result of this study is that timing beats volume. And **do not add a non-reverting constituent to sharpen the signal**: a machine whose every gear assumes reversion cannot be improved by feeding it something built not to revert – the argument is made in full in *The BuckBasket*, and the 2020-25 replay is the measurement.

8 Limitations and next increments

- The lazy catch-up is exact under sample-and-hold; a real gap has a price path. More pokes give higher fidelity at the same amortized price – and the basket's own activity supplies them precisely when flows (and thus fidelity needs) are high.
- The director reads whole-pool BUCK reserves; the venue nets out treasury liquidity. The small bias is irrelevant to hint *ordering*.
- Windows are governance parameters, seeded from the M2-lag analysis; the model says the premium is flat across 30-180d, so mis-tuning is forgiving.
- The basket-side consumption (increment 0 wiring in `BuckBasketProRata`, the `rebalanceSource/Sink` pair) is the next contract pass; the director itself, its tests, and the sim keeper are complete.
- Keeper bounty economics (a few bp of each productive step, paid from treasury) are needed on a public network and are not yet modeled.
- **The trend-regime pick has no on-chain engine.** `factor` wins the historical replay and is a Python policy only; the two Solidity engines are `vrata` and `pairs`. Either the regime split is accepted as a deliberate bet on reversion – which the basket's whole thesis already is – or `factor` needs an engine so governance can switch.
- The `pairs` basis is redundant (fifteen pairs over five independent modes at $N = 6$); see *Variants worth investigating*.